Lecture 0

01 October 2026 23:21

IMP reference codes are point.C.part eg., 2.C.1

1. For the transformer to trained from some data, we are feeding it with now a small data-set of Shakespeare text

Step 1: download a Shakespeare text, as a input.txt file

2.C.1. How can the data will be accessible to the transformer

```
with open("input.txt", "r", encoding="utf-8") as f:
    text = f.read()
```

with open is a code format to access a file data, with file name as "input.txt:, reading only hence r, and encoding means the data is already in coded form like in a form of numbers,
V IMP
we can understand it like this:
1. let's say the text is: "hello my name is" which is in encode form i.e. already coded in a form of number in machine learning
2. and the decoded form will be 32, 45, 67,4, 2, 4, 26 like that decoded form

utf-8 is a coding scheme, so that the data is converted into required bytes or bits

Source: Antigravity

Encoding vs. Decoding (Clearing up the direction)

Your idea about converting between text and numbers is spot on, but the terms **encoding** and **decoding** describe opposite directions:

- **Encoding (**$Text \rightarrow Numbers\ /\ Bytes$**":)"**
  - Takes human-readable text and translates it into bytes/numbers for the computer to store or send.
  - *Example:* Converting 'A' into the number 65 (or binary 01000001).
- **Decoding (**$Numbers\ /\ Bytes \rightarrow Text$**":)"**
  - Takes stored numbers/bytes and translates them back into human-readable text so you can read it on your screen.
  - *Example:* Converting the number 65 back into 'A'.

What is UTF-8 used for?

UTF-8 (*Unicode Transformation Format - 8-bit*) is the standard text encoding scheme used across the modern web (used by over 98% of websites) and operating systems. 3 benefits: 1. universal coverage, 2. Space efficiency, 3. Backward compatibility

2.C.2 reading the first 1000 characters of the text

```
print(text[:1000])
```

Output:
All the text upto to 1000 words will get printed

3.C.1 making the size of some things

```
chars = sorted(list(set(text)))
vocab_size = len(chars)
print("".join(chars))
print(vocab_size)
```

Here, the text is first combine or converted in a set, then, in a list, do that it will be in the format of ['a', 'b', 'c', etc], then it is sorted in order and given to chars,
V IMP

1. So, chars are the total characters used in the above 1000 words
2. vocab_size is the total size of the chars
3. And in last we have join all the chars used with each other without space in between (used of "".join(chars))

V IMP

3. Making Tokens

i.e. words should be converted into numbers for the machine to understand, and returns back the numbers to form a token and a new token

```
stoi = [ ch : i for i, ch in enumerate(chars) ]
itos = [ i : ch for i, ch in enumerate(chars) ]
encode = lambda s:  [ stoi[c] for c in s ]
decode = lambda l:  "".join( [ itos[i] for i in l ] )
```

stoi = is string to integer
itos = is integer to string
here

USE OF lambda
It's nothing but a def function, instead of writing whole def functions we can use the lambda function

```
print(encode("hii buddy") )
print(decode(encode("hii buddy") ) )
```

Output:
['27', '35', '35', '24', '26', …]
hii buddy

Video notes
Key points to remember for videos
For tokenization there are various Schemes like Sentence, etc where we can wite the encoding and the decoding

V IMP why do we need tokenizer to be very important
When a sentence/ line is given as input, it is further divided into many different small parts like this ['27', '35', '35', '24', '26', …]
But, if we have more access to character length or let's say size, we can convert that whole sentence into this ['27', '3542', 23']
i.e. only three numbers, so we get more tokens to use, as the size off the archicture has been reduced or can say modified, Hence its very important why each GPT models need modification and advancements.
Foe examples., TikTokens, Sentence, etc

Here, in this MODEL, we are using very small level character level tokenizer hence, we are getting a size of 65 of the vocab_size and the list of long integer values which are converted from a sentence

----------IMP insights for the use of CUDA env ----------

1. What's an cuda env and why do we use it for the GPT or the language model

CUDA is NVIDIA's technology that allows PyTorch and Python to run math on your **NVIDIA GPU (Graphics Card)** instead of your CPU.

- **CPU**: Built for general tasks, but only has 8 to 16 fast cores.
- **GPU (CUDA)**: Built for parallel math, containing **thousands of smaller cores**.
- **Why use it?** Deep learning models (like GPT) do millions of matrix multiplications. CUDA makes training **10x to 100x faster** than using a CPU.

V IMP LEARN:
In PyTorch, you send your data and model to CUDA like this:
device = 'cuda' if torch.cuda.is_available() else 'cpu'
data = data.to(device) # Moves data to GPU RAM

2. Does Jupyter Notebook NOT take code added before? How does it work?

Jupyter Notebook does **NOT** automatically update code just because you typed it in a cell.
Jupyter runs on **execution order (when you hit Shift+Enter)**, NOT cell position on your screen:

1. If you add encode = lambda ... in Cell 1, but **forget to run Cell 1**, Python's memory doesn't know encode exists yet.
2. If you then run Cell 2 (encode(text)), Python checks its memory, can't find encode, and throws NameError.

3. Why did "Restart Kernel and Run All" fix it, and do you have to do it every time?

Why it fixed the error:
When you clicked **Restart Kernel & Run All**:

1. It cleared out all old, confused variables from memory.
2. It executed your cells strictly from **top to bottom** (Cell 1 → Cell 2 → Cell 3).
3. Because Cell 1 (defining encode) ran *before* Cell 2 (using encode), everything worked smoothly!

Do you have to do "Restart Kernel" every time?
No, you do not have to restart every time.

- **When editing a cell above:** Simply click on that cell and press **Shift + Enter** to update Python's memory before running the cell below it.
- **When to use "Restart Kernel":** Only use it when variables get tangled/overwritten, or when GPU CUDA memory gets full and you want a fresh start.

4. Converting the entire text(the data-set) into an array here is Tensor, so that the GPT can look it

```
import torch

data = torch.tensor(encoding(data), dtype=tensor.long)
Print(data.shape, data.dtype)
Print(data[:1000])
```

Output:

```
torch.Size([1115394]) torch.int64
tensor([18, 47, 56, 57, 58,  1, 15, 47, 58, 47, 64, 43, 52, 10,  0, 14, 43, 44,
        53, 56, 43,  1, 61, 43,  1, 54, 56, 53, 41, 43, 43, 42,  1, 39, 52, 63,
         1, 44, 59, 56, 58, 46, 43, 56,  6,  1, 46, 43, 39, 56,  1, 51, 43,  1,
        57, 54, 43, 39, 49,  8,  0,  0, 13, 50, 50, 10,  0, 31, 54, 43, 39, 49,
         6,  1, 57, 54, 43, 39, 49,  8,  0,  0, 18, 47, 56, 57, 58,  1, 15, 47,
        58, 47, 64, 43, 52, 10,  0, 37, 53, 59,  1, 39, 56, 43,  1, 39, 50, 50,
         1, 56, 43, 57, 53, 50, 60, 43, 42,  1, 56, 39, 58, 46, 43, 56,  1, 58,
        ……] )
```

The whole text is encoded(converted into numbers) and store in an Tensor(array) with data-type tensor.long
i.e. its store In a single line of array

1. Tensor.shape = gives the size of the tensor,

IMP to know

2. What is torch.long used for?

In deep learning and NLP (Language Models like GPT), **torch.long is strictly required for integer IDs, indices, and class labels.**
Here are the main reasons why it is used:

1. Token IDs & Embedding Layers (The main reason in your code)
In language models, words or characters are converted into integer IDs (e.g., 'h' → 0, 'i' → 1).
When you pass these integer IDs into a PyTorch Embedding layer (nn.Embedding), PyTorch **requires** the tensor to be torch.long:

Python

```
# Token IDs must be torch.long
data = torch.tensor(encode("hii"), dtype=torch.long)                    IMP  tensor([0, 1, 1], dtype=torch.int64)

# Embedding layer looks up vectors using these long indices
embedding = torch.nn.Embedding(num_embeddings=65, embedding_dim=32)

out = embedding(data)  # Works smoothly!
```

(If you passed dtype=torch.float here, PyTorch would crash with an error).

5. Feeding the data to the transformer

i.e. splitting the data into train and validation set

```
n = int(0.9 * len(data) )
train_data = data[:n]
val_data = data[n:]
```

1.Why we took 0.9 = we will train the first 90 percent of the data from the text and remaining text will be there for validation
2.training data will range from 0(start) to n(that part i.e. 90 %)
3.the validation data will range from remaining n to last element

----------V V IMP know AND learn (get the knowledge)----------

1. What's the training data-set and validation data-set MEANS? And why we train only the 90% of the data and not the whole data at ONCE?
Imagine a student is preparing for an EXAM

1. Training Data (90%):
- The dataset the neural network directly reads, learns from, and uses to update its weights/parameters.
- **Analogy:** The **textbooks and practice exercises** the student uses to study.
- IMP Imagine the neural network as a student and the knowledge he know or is building from the resources is weights

2. **Validation Data (10%)**:
- A portion of the dataset that is kept secret. The model **never updates its weights** on this data. It is only used to test the model's accuracy on unseen text.
- **Analogy:** A **mock exam** with brand-new questions to check if the student actually understands the subject.

2. Why don't we use 100% of the data for training?

A. To Detect Overfitting (Memorization vs. Genuine Learning)
If you train a neural network on 100% of your data, it might simply **memorize** the exact sequences of text word-for-word instead of learning the underlying rules of language.
- If **Training Loss is low**, but **Validation Loss is high** → The model has overfitted (memorized).
- If **both Training and Validation Loss are low** → The model has actually learned to generalize.

B. Knowing When to Stop Training
By evaluating validation loss after each epoch/step, you can see when the model stops improving on unseen data and start overfitting. This tells you the exact moment to stop training.

What OVERFITTING?
If the model is just memorize the whole text it will just print the ouput as per the text contain.
Example: if a sentence end with "dancing like" and the data is having sentence like "dancing like bear", so it will print with that reference

So, it's important that our model does not OVERFIT and learn the patterns, HOW is this DONE?

By calculating the loss using (nn.Lossvalidation() functions).
IMP
Hence, our model should have the training and the validation loss low as possible

6. How the model will predict the next token
It will predict with the help of the previous all the token. NOTE: it will take help of al the previous token and not just one.
For example: h i i t h e r e encoding will be ['24', '26', '27', '82', '92', '37', '91', '62']
So, when it will take 1st index 24 and ----> predict 26 from the help of 24 --> 1 time
Similarly, for predicting 27 ----> it will take help of both 24, 26 --> 2 time

And so one upto block size + 1 times i.e. 9 times

6.C.1. what will be the code for it

```
block_size = 8
chunk = train_data[ : block_size+1 ]
print(chunk)
```

Output:

```
tensor([18, 47, 56, 57, 58,  1, 15, 47, 58])
```

here, we are training small small parts of the text, not all at once,
This helps the model to train itself and not to memorize.

7. How to achive the next prediction of the token

```
x = chunk[ : block_size ]
y = chunk[ 1 : block_size+1 ]
For t in range(block_size):
   context = x[ : t+1 ]
   target = y[ t ]
   print(f"when the input is {context} the target is: {target}")
```

output

```
when the input is tensor([18]) the target(output) is:47
when the input is tensor([18, 47]) the target(output) is:56
when the input is tensor([18, 47, 56]) the target(output) is:57
when the input is tensor([18, 47, 56, 57]) the target(output) is:58
when the input is tensor([18, 47, 56, 57, 58]) the target(output) is:1
when the input is tensor([18, 47, 56, 57, 58,  1]) the target(output) is:15
when the input is tensor([18, 47, 56, 57, 58,  1, 15]) the target(output) is:47
when the input is tensor([18, 47, 56, 57, 58,  1, 15, 47]) the target(output) is:58
```

1. x is input:
hence we will take it from the starting index let's say to the block_size i.e. 8 because the 9th index will be the one predicted by the previous one,
NOTE : if we take block_size + 1 as input --> then the last 9th element is also taken as input and the model tries to predict the 10th element

2. And for the output/ Target
Y (target) = we will take from the one ahead index till the last plus one
As the 1 is for the predicted one and the last one is the 9th element which is bloack size + 1

V IMP
Why do we take only the block_size as 8 and not HIGHER?
Cause, it enough for the model to train for

[----------in short this is just for the printing/ or letting the model know how to get the block or chunk of the text data ----------]

all this help in later Inference concept

8. how
Then we will have many chunks of data for training

We need all this chunks to be feed together to the model, and as there are all independent so they will not interact with each other

We will pack this all chunks of data in a tensor just as we did for the whole text (which was converted into tensor named data)
Because the GPU is more efficient in anayalizing all this data in parallel